



AUDIT REPORT

September 2026

For

xhayic

Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
■ High Severity Issues	14
1. minTxAmount can lock low-decimal ERC20 tokens because it is decimal-agnostic	14
■ Medium Severity Issues	15
2. Missing non-standard ERC20 token handling	15
3. ERC20 execution can silently short-deliver and emit misleading events	16
4. Unclaimed platform fees are spendable by the wallet owner	18
■ Low Severity Issues	19
5. Platform pause can freeze withdrawals indefinitely and make users lose on fees	19
6. Factory ownership registry becomes permanently stale after transferOwnership	20
7. Native fee-failure handling is inconsistent and leaves dead accumulator logic	21
■ Informational Severity Issues	22
8. Security critical delay is hardcoded to test value of 2 minutes	22
9. platformAdmin updates can be two-step	23
10. No events emitted for admin state changes	24
11. Missing test suite	25
Functional Tests	26
Threat Model	27
Automated Tests	33
Closing Summary & Disclaimer	34

Executive Summary

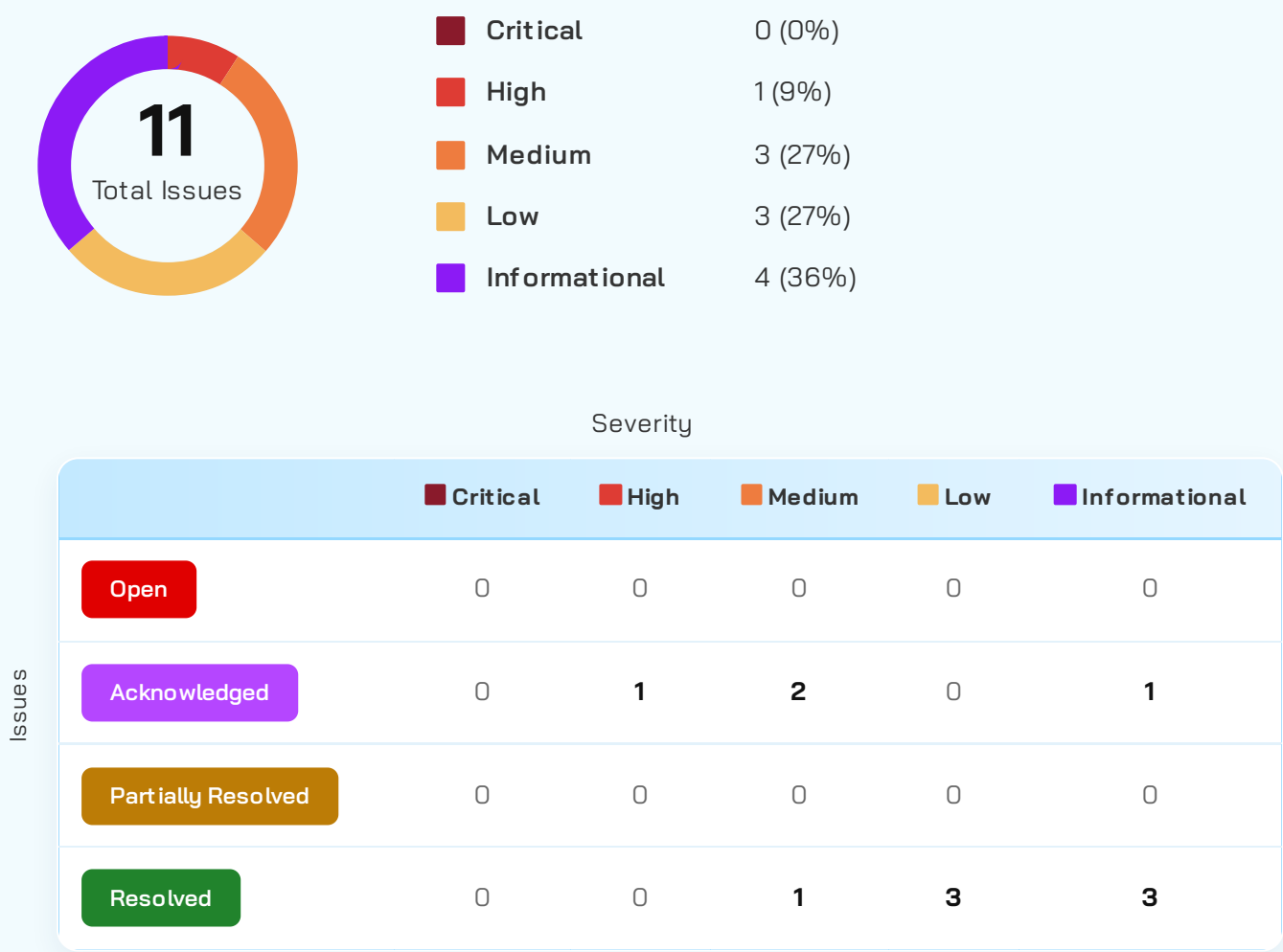
Project Name	TimeDelay Wallet
Protocol Type	Time-locked wallet
Project URL	https://xhavic.com/
Overview	TimeDelayWallet is a clone deployed from the WalletFactory which introduces a queue-and-execute transfer with a mandatory delay and platform fees collected.
Audit Scope	The scope of this audit was to analyze the TimeDelay Wallet Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/parmanand-carpenter/Time-Secure_wallet_Blockchain/tree/main/contracts
Contracts in Scope	contracts/TimeDelayWallet.sol contracts/WalletFactory.sol
Branch	main
Commit Hash	e93b34
Language	Solidity
Blockchain	xHavic, Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	20th - 22nd April, 2026
Updated Code Received	17th September, 2026
Review 2	17th - 18th September, 2026
Fixed In	5e8d1a5

Executive Summary

✓ Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	minTxAmount can lock low-decimal ERC20 tokens because it is decimal-agnostic	High	Acknowledged
2	Missing non-standard ERC20 token handling	Medium	Acknowledged
3	ERC20 execution can silently short-deliver and emit misleading events	Medium	Acknowledged
4	Unclaimed platform fees are spendable by the wallet owner	Medium	Resolved
5	Platform pause can freeze withdrawals indefinitely and make users lose on fees	Low	Resolved
6	Factory ownership registry becomes permanently stale after transferOwnership	Low	Resolved
7	Native fee-failure handling is inconsistent and leaves dead accumulator logic	Low	Resolved
8	Security critical delay is hardcoded to test value of 2 minutes	Informational	Resolved
9	platformAdmin updates can be two-step	Informational	Resolved
10	No events emitted for admin state changes	Informational	Resolved
11	Missing test suite	Informational	Acknowledged

Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls

✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Style guide violation

✓ Using throw

✓ Unsafe type inference

✓ Using inline assembly

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Hardhat, Solhint, Mythril, Slither, Solidity statistic analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open Security vulnerabilities identified that must be resolved and are currently unresolved.	Resolved These are the issues identified in the initial audit and have been successfully fixed.
Acknowledged Vulnerabilities which have been acknowledged but are yet to be resolved.	Partially Resolved Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		■ High	■ Medium	■ Low
Likelihood	■ High	Critical	High	Medium
	■ Medium	High	Medium	Low
	■ Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

minTxAmount can lock low-decimal ERC20 tokens because it is decimal-agnostic

Acknowledged

Path

```
contracts/TimeDelayWallet.sol
```

Function Name

```
queueTransaction()
```

Description

`minTxAmount` is initialized to `1e13` and enforced for all assets. This threshold is implicitly tuned for 18-decimal assets, but is applied to all ERC20 tokens. For common low-decimal tokens:

- 6 decimals (USDC-style): minimum becomes `10,000,000 USDC`.
- 8 decimals: minimum becomes `100,000 USDC` tokens.

If users deposit smaller amounts, they cannot queue withdrawals because `_value >= minTxAmount` fails.

Impact

Medium, this is a direct availability risk.

Likelihood

High

Proof of Concept

1. User deposits `1,000 USDC` (6 decimals) into a wallet clone.
2. User calls `queueTransaction(recipient, 1_000e6, usdcAddress)`.
3. Transaction reverts.

Recommendation

Use a per-asset minimum, instead of a single global threshold.

Medium Severity Issues

Missing non-standard ERC20 token handling

Acknowledged

Path

`contracts/TimeDelayWallet.sol`

Function Name

`queueTransaction()`

Description

The contract logic assumes standard ERC20 transfer semantics. Fee-on-transfer/rebasing/deflationary behavior can cause balance drift with lockedAmount which will lead to reverts and confirmed operational lockups. Also, if the `platformAdmin` address is blocklisted by a token issuer (e.g., Circle for USDC), the `safeTransfer(platformAdmin, fee)` call at L192 will revert, permanently DoS-ing all `queueTransaction()` calls for that specific token. The owner cannot change the platformAdmin. The only escape is if platformAdmin calls `updateFee(0)`.

Impact

Medium

Likelihood

Medium, as users determine what tokens they want locked.

Recommendation

Gate using an allowlist or handle ERC20 token transfers with the available balance in the wallet, not the stored `_value` amount.

Medium Severity Issues

ERC20 execution can silently short-deliver and emit misleading events

Acknowledged

Path

```
contracts/TimeDelayWallet.sol
```

Function Name

```
executeTransaction()
```

Description

For ERC20 transfers, `executeTransaction()` sends the lesser of the queued amount and the wallet's entire current token balance:

```
uint256 available = IERC20(txn.token).balanceOf(address(this));
uint256 toSend = txn.value > available ? available : txn.value;
IERC20(txn.token).safeTransfer(txn.to, toSend);
```

This creates three issues:

1. A transaction may be marked as executed even when the recipient receives less than the queued amount.
2. `TransactionExecutedDetailed` emits `txn.value`, not the actual `toSend` amount.
3. The full wallet balance is considered available, including funds backing other pending transactions and unclaimed fees.

If the transfer succeeds with a reduced amount, the transaction cannot be retried or cancelled because it has already been finalized.

If the token transfer itself reverts, the entire transaction reverts and the state changes are rolled back. The issue applies when the transfer succeeds but delivers less than the intended amount.

Proof of Concept

1. Owner holds a rebasing token; queues Tx A (1,000) and Tx B (1,000). `LockedAmount = 2,000`, balance 2,000.
2. A negative rebase reduces the wallet balance to 1,600.
3. Owner executes Tx A: recipient receives `min(1000, 1600) = 1,000`. Balance 600.
4. Owner executes Tx B: recipient receives `min(1000, 600) = 600`. Event emits 1,000. Tx B is marked executed; the 400 shortfall is unrecorded and unrecoverable.

ERC20 execution can silently short-deliver and emit misleading events

(continued)

Recommendation

- Emit the actual delivered amount (add a `shortfall` or `amountSent` field to the event).
- Make short-delivery an explicit decision rather than a silent default
- Bound `available` by excluding other transactions' locked value: `available = balanceOf(this) - lockedAmount[token]` (computed after this transaction's own decrement), so one transfer cannot consume another's backing.

Medium Severity Issues

Unclaimed platform fees are spendable by the wallet owner

Resolved

Path

```
contracts/TimeDelayWallet.sol
```

Function Name

```
queueTransaction(), getAvailableBalance(), claimFees()
```

Description

When an ERC20 fee transfer to `platformAdmin` fails, the fee is accumulated in `unclaimedFees[_token]`. However, the accumulated amount is not included in the wallet's reserved balance calculations.

The queueing logic and `getAvailableBalance()` account only for `lockedAmount[_token]`:

```
balance - lockedAmount[_token]
```

The correct solvency invariant should be:

```
balanceOf(wallet) >= lockedAmount[token] + unclaimedFees[token]
```

Because `unclaimedFees[_token]` is omitted, the wallet owner can queue transfers using tokens that are owed to the platform.

Recommendation

Include unclaimed fees in every reservation and availability calculation and apply the same reservation logic in `getAvailableBalance()`.

Low Severity Issues

Platform pause can freeze withdrawals indefinitely and make users lose on fees

Resolved

Path

`contracts/TimeDelayWallet.sol`

Function Name

`executeTransaction()`

Description

`setPaused(true)` blocks both queue and execute functionality. This lets platform admin halt all withdrawals, not just new queues. If the admin key is lost/compromised, funds can remain frozen indefinitely – or the user will have to call `cancelTransaction()` to retrieve their funds but will compulsorily take a haircut because fees are not refundable.

Impact

Medium, temporary DoS on withdrawals across affected wallet clone(s).

Likelihood

Low.

Recommendation

The 24 hour delay should be sufficient to cancel mistaken transfers but not restrict all other users from executing pending transactions.

Low Severity Issues

Factory ownership registry becomes permanently stale after transferOwnership

Resolved

Path

```
contracts/WalletFactory.sol, contracts/TimeDelayWallet.sol
```

Function Name

```
transferOwnership()
```

Description

`_walletOwner[clone]` and `_wallets[owner]` are written once in `createWallet` and never updated when `TimeDelayWallet.transferOwnership` / `acceptOwnership` changes the actual owner of a wallet.

`WalletFactory` records the wallet creator in `_walletOwner[clone] = msg.sender` and `_wallets[msg.sender].push(clone)` at creation time. `TimeDelayWallet` exposes a two-step ownership transfer (`transferOwnership` + `acceptOwnership`) that moves the owner on the clone but never notifies the factory.

Impact

Integrators and UIs that trust the factory's view functions as an authoritative owner directory will misattribute wallets after any ownership transfer. The registry is fundamentally inconsistent with the source of truth on each clone.

Likelihood

High.

Recommendation

Update both mappings when ownership changes.

Low Severity Issues

Native fee-failure handling is inconsistent and leaves dead accumulator logic

Resolved

Path

`contracts/TimeDelayWallet.sol`

Function Name

`queueTransaction(), claimFees()`

Description

ERC20 fee transfers use the `unclaimedFees` fallback when the transfer fails. Native fee transfers still revert when the platform admin cannot receive native currency:

```
(bool feeSent, ) = platformAdmin.call{value: fee}("");  
require(feeSent, "Fee transfer failed");
```

As a result, a non-payable or reverting `platformAdmin` can block all native-coin queues. The native branch of `claimFees()` is effectively unreachable because no code path increments:

```
unclaimedFees[address(0)]
```

Impact

A misconfigured or non-payable platform admin can cause a denial of service for native-asset queues. The fee accumulator has inconsistent behavior between native and ERC20 assets, increasing maintenance and integration risk.

Recommendation

Use the same failure-handling model for native fees:

```
(bool feeSent, ) = platformAdmin.call{value: fee}("");  
  
if (!feeSent) {  
    unclaimedFees[address(0)] += fee;  
}
```

Informational Severity Issues

Security critical delay is hardcoded to test value of 2 minutes

Resolved

Path

contracts/TimeDelayWallet.sol

Function Name

queueTransaction(), DELAY

Description

This is documented in the README and within the codebase, but flagged to ensure it is not missed.

Recommendation

Implement the update to `24 HOURS` in production.

Informational Severity Issues

platformAdmin updates can be two-step

Resolved

Path

`contracts/TimeDelayWallet.sol`

Function Name

`updatePlatformAdmin()`

Description

Fees are sent to the `platformAdmin` and will cost the platform all fees generated from the users' wallets. Currently, the `platformAdmin` address can be updated by the existing `platformAdmin` but is only restricted from being `address(0)` – this could cause a mistaken transfer of ownership.

Recommendation

Make this a two-step approach similar to Ownable.

Informational Severity Issues

No events emitted for admin state changes

Resolved

Path

```
contracts/TimeDelayWallet.sol
```

Function Name

```
setPaused(), updateFee(), updatePlatformAdmin()
```

Description

Three critical admin functions modify protocol state without emitting any events:

- `setPaused()` (L262): `paused = _paused;` — no event
- `updateFee()` (L288): `feeBps = _newFeeBps;` — no event
- `updatePlatformAdmin()` (L294): `platformAdmin = _newPlatformAdmin;` — no event

A compromised platformAdmin could silently change fees to 5%, pause the wallet, and transfer admin rights without any on-chain event trail.

Impact

Off-chain monitoring systems cannot detect admin abuse.

Recommendation

Add events:

```
event PauseToggled(bool paused)
event FeeUpdated(uint256 oldFeeBps, uint256 newFeeBps)
event PlatformAdminUpdated(address oldAdmin, address newAdmin)
```

Informational Severity Issues

Missing test suite

Acknowledged

Description

The protocol has no unit, integration or fuzz tests.

Functional Tests

- ✓ Wallet creation and registration
- ✓ Correct wallet owner initialization
- ✓ Correct platform-admin initialization
- ✓ Wallet discovery through getWallets()
- ✓ Wallet count tracking through getWalletCount()
- ✓ Current ownership validation through getWalletOwner()
- ✓ Current ownership validation through isWalletOf()
- ✓ Ownership transfer reflected immediately in factory views
- ✓ Unauthorized transaction queueing is rejected
- ✓ Unauthorized pause operations are rejected
- ✓ Unauthorized fee updates are rejected
- ✓ Unauthorized platform-admin updates are rejected
- ✓ Re-initialization attempts are rejected
- ✓ Two-step wallet ownership transfer
- ✓ Two-step platform-admin transfer
- ✓ Pausing blocks new transaction queueing
- ✓ Pausing does not block execution of queued transactions
- ✓ Pausing does not block transaction cancellation

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

Dependency	Type	How Used	Trust Assumption	Risks
OpenZeppelin Clones	Library	WalletFactory deploys EIP-1167 clones of the implementation	Clone bytecode and delegate behaviour are correct and deterministic	If the implementation address is wrong/malicious at factory deployment, all cloned wallets inherit unsafe logic.
OpenZeppelin SafeERC20	Library	Used for ERC20 fee and execution transfers	Wrapper behaviour is correct and token returns are handled safely	Non-standard tokens (fee-on-transfer/rebasing) can break economic assumptions.
Solidity Compiler	Tool	Compilation	Compiler has no known critical bugs	Older version can lead to unsafe defaults

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

Contract Name	Function	Category
TimeDelayWallet	initialize()	Main invariant(s) initialized flips once from false to true per clone. Access level external (one-time gate via initialized). What this function can do Initialize clone owner/admin and default economic parameters once. What this function cannot/should not do Cannot run twice; should not accept zero addresses.
TimeDelayWallet	receive()	Main invariant(s) Raw balance may increase without changing lockedAmount. Access level external payable, open to all. What this function can do Accept native coin deposits from any sender. What this function cannot/should not do Must not mutate queue/lock state.
TimeDelayWallet	queueTransaction(address _to, uint256 _value, address _token)	Main invariant(s) lockedAmount[token] += value; new tx starts executed=false and cancelled=false. Access level onlyOwner, nonReentrant, notPaused. What this function can do Create queued transfer, snapshot fee, increment txCounter, increase lock, transfer fee to platform admin. What this function cannot/should not do Cannot queue below minimum, to zero recipient, or beyond available unlocked balance.

Contract Name	Function	Category
TimeDelayWallet	executeTransaction(uint256 _txId)	Main invariant(s) At-most-once execution per tx; lockedAmount[token] -= value before transfer. Access level onlyOwner, nonReentrant, notPaused. What this function can do Execute matured queued transfer and send value to recipient. What this function cannot/should not do Cannot execute invalid, cancelled, already executed, or not-yet-matured tx.
TimeDelayWallet	cancelTransaction(uint256 _txId)	Main invariant(s) At-most-once cancellation per tx; unlock amount equals queued value. Access level onlyOwner (works while paused). What this function can do Cancel queued tx and unlock reserved value. What this function cannot/should not do Cannot cancel invalid, executed, or already cancelled tx.
TimeDelayWallet	setPaused(bool _paused)	Main invariant(s) Pause gates queue/execute paths in current design. Access level onlyPlatform. What this function can do Toggle paused state. What this function cannot/should not do Must not directly move funds or alter queued tx metadata.

Contract Name	Function	Category
WalletFactory	createWallet()	Main invariant(s) _walletOwner[clone] == msg.sender; clone added to caller wallet list. Access level external, open to all. What this function can do Deploy wallet clone, initialize it, store owner mappings, emit creation event. What this function cannot/should not do Cannot assign arbitrary owner/admin inputs beyond caller and immutable platform admin.

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

Contract	Asset Type	Custodied Assets
TimeDelayWallet.sol	Native tokens, ERC20	User deposited balances held in wallet contract storage until execution or cancellation path.

3.2 Asset Entry & Exit Functions

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TimeDelayWallet.sol	queueTransaction(..., _token = address(0))	-	Native fee to platformAdmin	owner only	If platformAdmin is misconfigured/inaccessible, fees are irrecoverable; fee transfer revert blocks queue.
TimeDelayWallet.sol	queueTransaction(..., _token != address(0))	-	ERC20 fee to platformAdmin	owner only	Fee-on-transfer/rebasing tokens can violate expected fee/value economics and create execution friction.
TimeDelayWallet.sol	executeTransaction(..., token = address(0))	-	Native txn.value to txn.to	owner only	notPaused gate can block exits while paused; recipient revert blocks this tx until cancel.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TimeDelayWallet.sol	executeTransaction(..., token != address(0))	-	ERC20 txn.value to txn.to	owner only	Non-standard ERC20 behavior (fee-on-transfer/rebasing/blacklist tokens) can cause short-delivery or execution revert patterns.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of TimeDelay Wallet. We performed our audit according to the procedure described above.

1 High, 3 Medium, 3 Low and 4 Informational severity issues were found.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For

xhayic



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com